



POSTECH

FIAP X — Processador de Vídeos

Resumo do Projeto · Tech Challenge Fase 5 · Hackathon

github.com/tgreimi/fiapx

FIAP X — Processador de Vídeos

Resumo do Projeto · Tech Challenge (SOAT) · Fase 5 — Hackathon

1. O Projeto

O FIAP X é um sistema de processamento de vídeos: a partir do upload de um vídeo, o sistema extrai frames e devolve um arquivo **.zip** para download. O desafio consistiu em re-arquitar o projeto base (um monolito Go, o *projeto-fiapx*) em microsserviços, aplicando boas práticas de arquitetura de software: desenho de arquitetura, mensageria, qualidade de software, observabilidade e CI/CD.

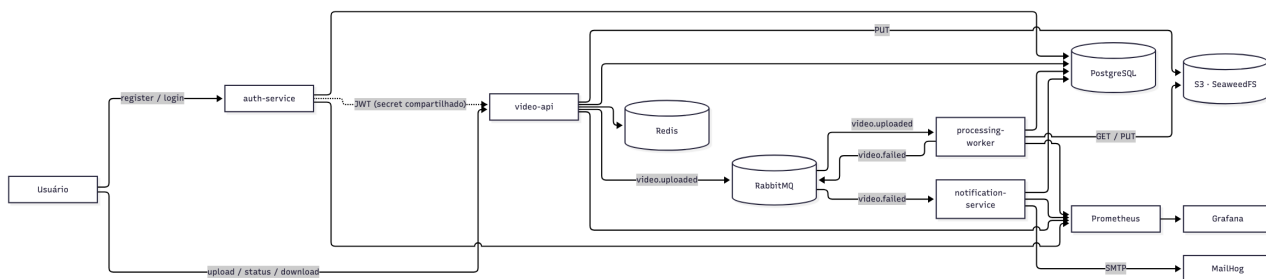
2. Arquitetura

- **Microsserviços + Arquitetura Orientada a Eventos** (coreografia via RabbitMQ) + **Database-per-Service**.
- Estilo interno **Hexagonal (Ports & Adapters)** em cada serviço.
- Módulos Go **independentes** em monorepo (um *go.mod* por serviço, zero imports cruzados).
- Comunicação entre serviços exclusivamente por eventos/HTTP.

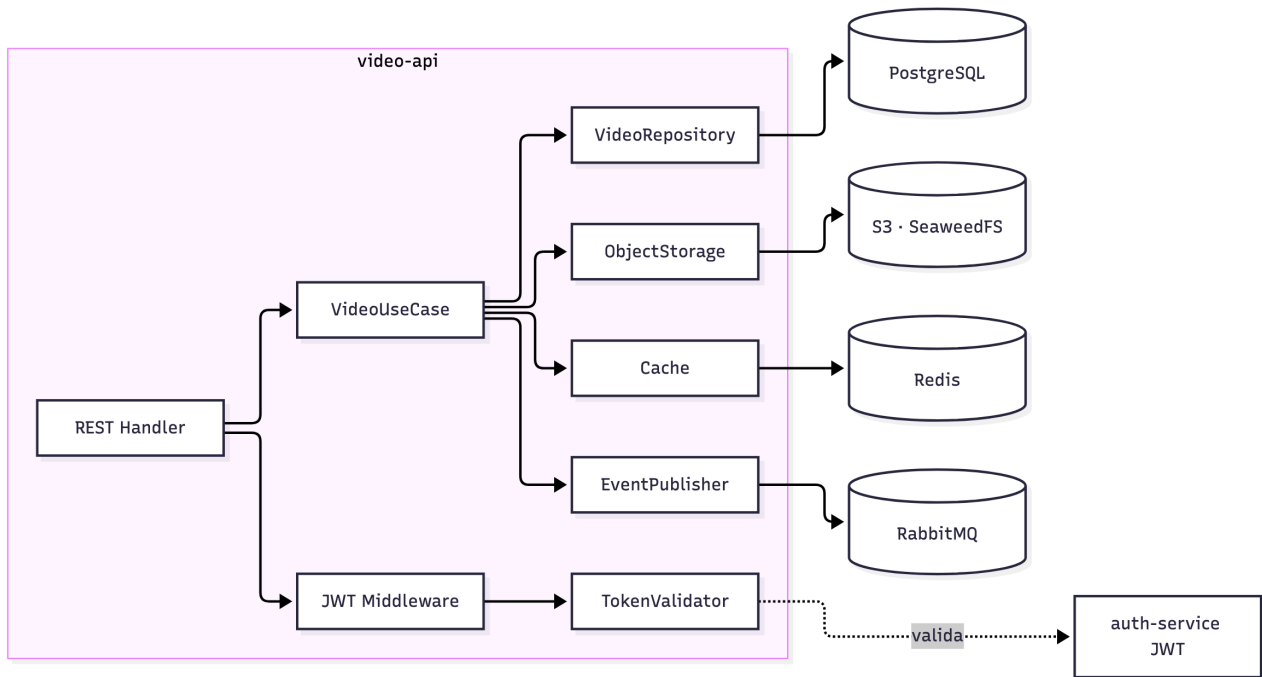
3. Diagramas C4



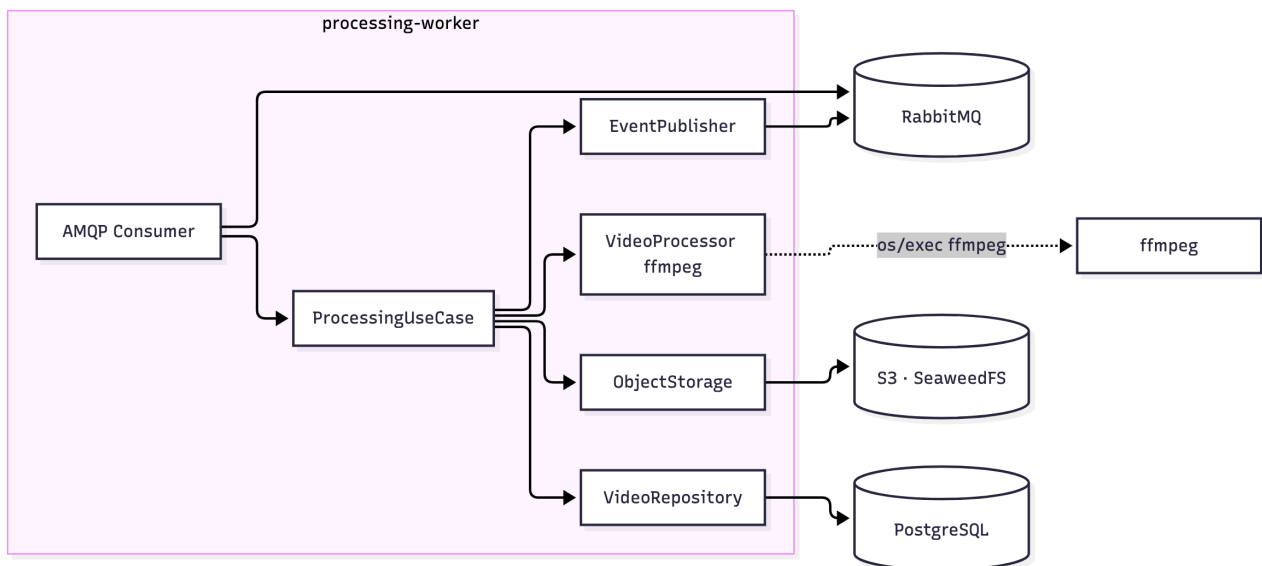
Nível 1 — Contexto



Nível 2 — Container



Nível 3 — Componente (video-api)



Nível 3 — Componente (processing-worker)

4. Serviços

Serviço	Porta	Responsabilidade
auth-service	8081	Cadastro/login e emissão de JWT
video-api	8080	Upload, listagem de status por usuário, download
processing-worker	— (métricas 9090)	Consume a fila, roda ffmpeg, zipa frames, salva no storage, atualiza st
notification-service	— (métricas 9091)	Consume evento de erro e envia e-mail

5. Stack Tecnológica

- **Linguagem:** Go (1.25+/1.27).
- **Containers/orquestração:** Docker + Docker Compose e manifests Kubernetes (*infra/k8s/*).
- **Mensageria:** RabbitMQ (com filas de retry com TTL e DLQ).
- **Dados:** PostgreSQL (persistência) + Redis (cache).
- **Storage:** S3 via SeaweedFS (dev) com adaptador *FilesystemStorage* como fallback.
- **Observabilidade:** Prometheus + Grafana (dashboards provisionados).
- **CI/CD:** GitHub Actions, uma esteira por serviço.

6. Resiliência e Qualidade

- **Resiliência:** fila *.retry* com TTL + dead-letter, header *x-retry-count*, *maxRetries=3*, DLQs e idempotência no consumo.
- **Testes:** unitários (rápidos, sem Docker) e de integração via Testcontainers (PostgreSQL/Redis/RabbitMQ reais).
- **Lint:** golangci-lint em todas as esteiras.

7. Observabilidade — Métricas

Cada serviço expõe métricas Prometheus em **/metrics**. Para consultar, use `make metrics` (exibe os valores atuais) ou acesse as URLs abaixo:

Serviço	Endpoint	Métricas de negócio
auth-service	:8081/metrics	auth_registrations_total, auth_logins_total
video-api	:8080/metrics	videos_uploaded_total, videos_downloaded_total
processing-worker	:9092/metrics	videos_processed_total, video_processing_errors_total, video_processing_duration_total
notification-service	:9093/metrics	emails_sent_total, emails_failed_total

Os serviços HTTP (*auth-service* e *video-api*) também expõem *http_requests_total* e *http_request_duration_seconds*. O Grafana (*admin/admin* em `:3000`) já vem provisionado com o dashboard *fiapx.json*.

8. Entregáveis

- Documentação da arquitetura: ADRs (001–005) e diagramas C4 (*docs/*).
- Script de criação do banco (*infra/db/init.sql*).
- Manifests Kubernetes e CI/CD por serviço.
- Coleção Bruno com as rotas documentadas (*bruno/*).
- Vídeo de apresentação (≤ 10 min): https://drive.google.com/file/d/1rCsacDJJqFiqJowYa9BUbf_oc6do7oHY/view

9. Repositório

Repositório do projeto
<code>https://github.com/tgremi/fiapx</code>